

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS



SEDE: SANTA CRUZ DEL QUICHÉ
CURSO: **PREPRIVADO**
Docente: ING. IVER ADOLFO VARGAS VILLANUEVA

PREPRIVADO
"PRÁCTICA DE ANÁLISIS Y DESARROLLO DE SISTEMAS"

NOMBRE: ISMAEL ENRIQUE
CORTEZ SUTUC
CARNÉ: 1090-22-19454

SANTA CRUZ DEL QUICHÉ, 18 de noviembre de 2025

Objetivo

Definir requisitos funcionales, no funcionales y criterios de aceptación para el software que permita gestionar ventas, pagos y reportes de los sorteos "La Santa", "La Rifa" y "El Sorteo".

El objetivo es diseñar e implementar un sistema para la tienda "La Suerte" que gestiona la venta de números de apuestas, el control de sorteos, la recaudación y el pago de premios.

1- Resumen del caso

“La Suerte” vende números (00–99) para tres sorteos distintos (La Santa, La Rifa, El Sorteo) con distintas tasas de pago. Se requiere sistema para registrar apuestas, imprimir voucher, controlar recaudaciones por sorteo/evento/día, identificar ganadores, aplicar bonificación del 10% si el cliente está de cumpleaños, y controlar plazo de cobro de premios (5 días hábiles).

2 — Análisis de Requerimientos

Requerimientos funcionales (RF)

- RF1. Registrar clientes (nombre, DNI/CC, teléfono, dirección, fecha de nacimiento).
- RF2. Registrar ventas/apuestas: cliente (opcional), sorteo, fecha/hora, turno (evento número dentro del día, ej. Santa 1/2/3), número (00–99), monto apostado, cajero/atendiente, imprimir voucher con datos.
- RF3. Permitir venta de un mismo número a varias personas (no exclusividad).
- RF4. Registrar sorteos diarios y resultados (número ganador por sorteo-evento).
- RF5. Calcular y pagar premio según tipo de sorteo: Santa Q25/Q1, Rifa Q70/Q1, Sorteo Q150/Q1; aplicar +10% si cliente cumple años en la fecha del sorteo.
- RF6. Control de vigencia para cobro: permitir cobro hasta 5 días hábiles desde la fecha del sorteo.
- RF7. Generar listado de ganadores al final del día; si no hay ganador en un evento, mostrar “ganador desierto”.
- RF8. Reportes: recaudación por día, por rango de fechas, por sorteo, por evento (ej. Santa 1 de X fecha), listados de ventas por cajero, vouchers.
- RF9. Autenticación básica de usuarios (cajero/administrador).
- RF10. Exportar/Imprimir reportes (PDF/impresora).

Requerimientos no funcionales (RNF)

- RNF1. Interfaz simple, usable en pantalla táctil o PC (mockups responsivos).
- RNF2. Rendimiento: registrar apuesta <1s en condiciones normales.
- RNF3. Disponibilidad: sistema básico local (single-server) o web interna.
- RNF4. Seguridad: roles (cajero, admin), registrar quien atendió cada venta, backups diarios de DB.
- RNF5. Mantenimiento: documentación técnica y manual de uso.
- RNF6. Persistencia: base de datos relacional (MySQL/MariaDB o PostgreSQL).

RNF7. Impresora de tickets compatible con impresión desde el navegador o con librería en servidor.

3 — Reglas de Negocio (RN)

RN1. Tipos de sorteo y factor de pago:

- La Santa → paga Q25 por Q1 (factor 25).
- La Rifa → paga Q70 por Q1 (factor 70).
- El Sorteo → paga Q150 por Q1 (factor 150).

RN2. Frecuencia diaria: La Santa → 3 eventos/día; La Rifa → 1 evento/día; El Sorteo → 2 eventos/día.

RN3. Números válidos: 00–99. Un mismo número puede venderse a N clientes por evento.

RN4. Bonificación: Si la fecha del sorteo coincide con fecha de nacimiento del cliente (día y mes), premio = premio * 1.10.

RN5. Plazo de cobro: 5 días hábiles desde la fecha del sorteo. Después se considera “no reclamado”.

RN6. Voucher: debe imprimirse en el momento de la compra con datos: cliente (si lo registra), número, sorteo+evento, monto apostado, fecha/hora, cajero, ID de voucher.

RN7. Si al finalizar un evento no hay apuestas ganadoras → mostrar “ganador desierto”.

RN8. Reportes deben poder filtrar por fecha, sorteo, evento (ej. Santa - evento 1), cajero.

4 — Arquitectura de Software sugerida

- Arquitectura: **3 capas** (Presentación — Lógica de negocio — Persistencia).
- Tipo de implementación: **Aplicación web** (servidor local o VPS) con frontend en HTML/CSS/JS (o framework ligero) y backend en PHP (Laravel), Node.js (Express) o Python (Flask/Django). Base de datos MySQL/MariaDB o PostgreSQL.
- Motivo: facilita despliegue en tienda y generación de reportes/impresión desde navegador.

Diagrama lógico (texto):

- Cliente (navegador) ↔ Web Server (API) ↔ DB (MySQL)
- Opcional: impresora térmica conectada al PC/servidor.

5 — Modelo de desarrollo y metodología

- **Modelo:** Iterativo incremental (ej. 3 iteraciones / entregas).
- **Metodología:** Scrum-lite o Kanban. Para entregables académicos recomiendo un cronograma con sprints semanales.
- Fases: análisis → diseño → implementación prototipo básico → pruebas → entrega.

6 — Análisis de Factibilidad

- Técnica: Alta. Requisitos típicos de CRUD y reportes; se puede implementar con herramientas libres (LAMP).
- Económica: Baja inversión (PC + impresora), software open-source.
- Operacional: Alto — personal capacitado con formación básica puede operar.
- Legal: Manejo de datos personales debe respetar privacidad local; almacenar mínimo necesario (teléfono, nombre, D.N.I. opcional).

7 — Planificación sugerida (cronograma — ejemplo 5 Días)

Sprint 1 (Día 1–1): Requerimientos y diseño — DERCAS, diagramas UML (caso de uso, actividades), DB diseño y script inicial.
 Sprint 2 (Día 2–2): Implementación prototipo: registro clientes, ventas, voucher, login, impresión básica.
 Sprint 3 (Día 3-3): Reportes, registro resultados de sorteos, cálculo de premio con regla de cumpleaños, controles de vencimiento de cobro, pruebas y documentación final.

8 — Modelo de datos (ER) — descripción y script SQL

Entidades principales

- customers (clientes)
- users (empleados: cajeros/admin)
- raffles (tipos de sorteo: Santa, Rifa, Sorteo)
- events (cada sorteo-evento por día; ej. Santa evento #1 del 2025-11-09 18:00)
- bets (apuestas/ventas)
- raffle_results (resultado por event)
- payments (registro de pago de premios reclamados)
- vouchers (opcional: registro voucher y código)
- [Diagrama ER \(texto\)](#)
- customers 1—N bets
- users 1—N bets
- raffles 1—N events
- events 1—N bets
- events 1—1 raffle_results (cuando se publica resultado)

9 — Cálculo de premio (lógica)

Para cada apuesta ganadora:

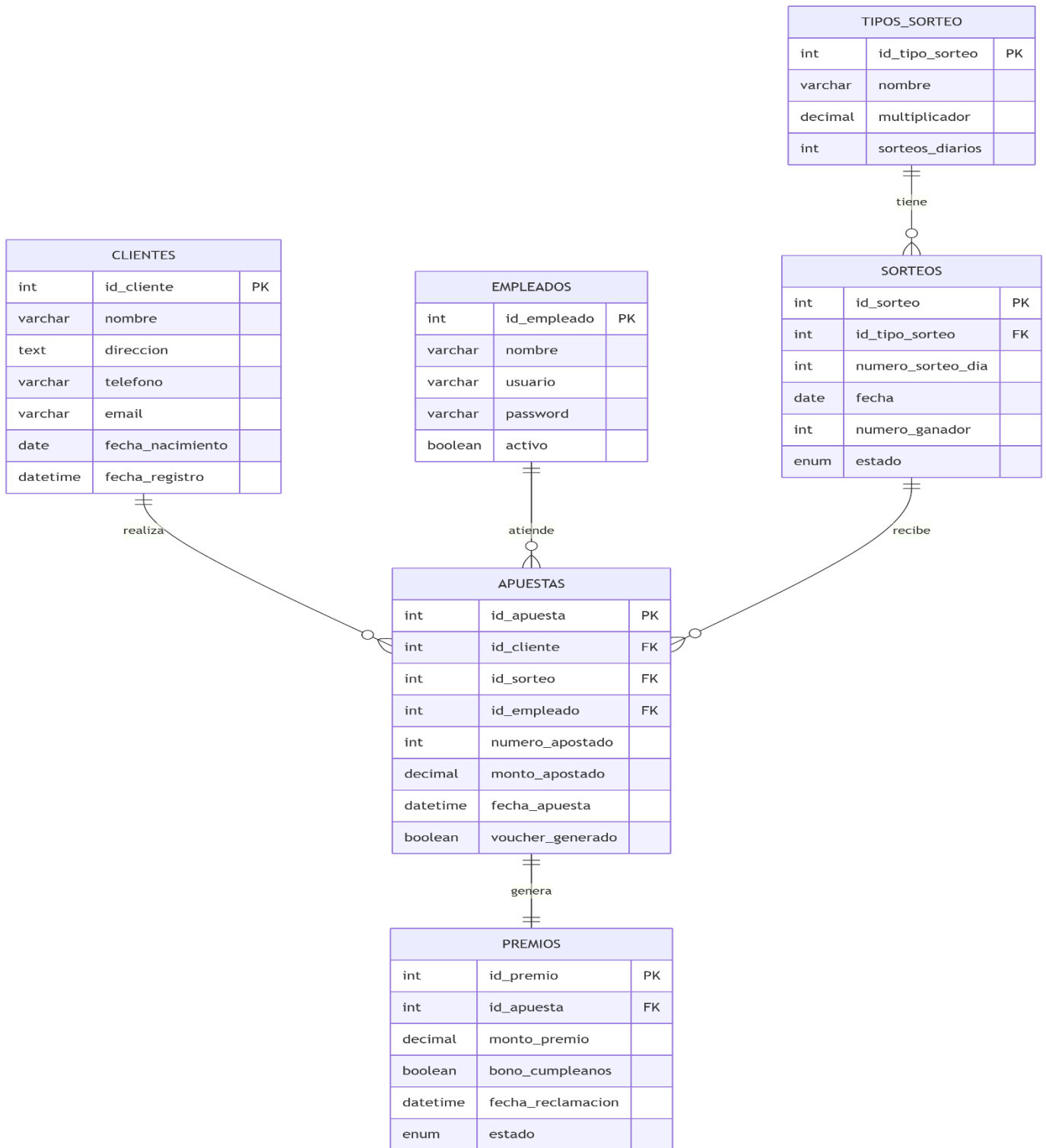
- `premio_base = amount * pay_factor`
- if cliente cumple años ese día (comparar día y mes de `birth_date` con `event_date`) → `premio = premio_base * 1.10`
- Retener registro con `paid_amount` cuando se pague.
- Comprobar vigencia: fecha límite = sumar 5 días hábiles a `event_date` (excluir sáb/dom? depende: aquí se define como días hábiles — excluye sáb-dom). Puedo incluir función para calcular 5 días hábiles en SQL o en backend.

Ejemplo en pseudocódigo:

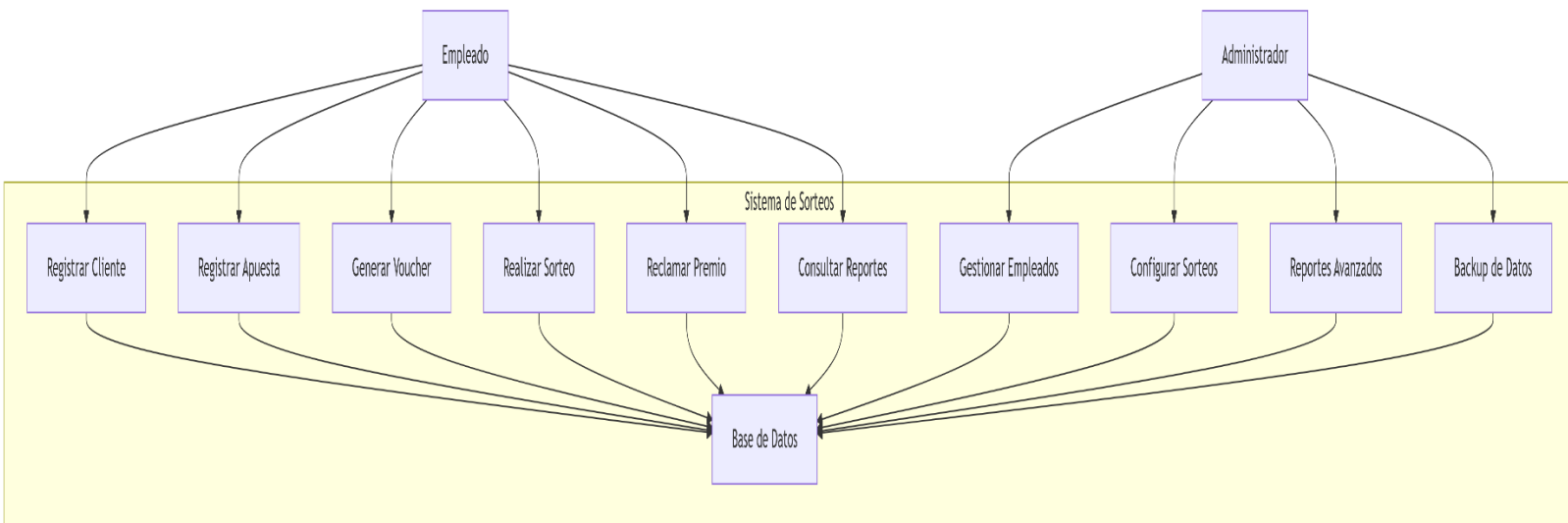
```
if (bet.number == winning_number) {  
    prize = bet.amount * raffle.pay_factor;  
    if (customer.birth_date DAY/MONTH == event_date DAY/MONTH) prize *= 1.10;  
}
```

10 —Diagramas del Sistema de Control de Sorteos "La Suerte"

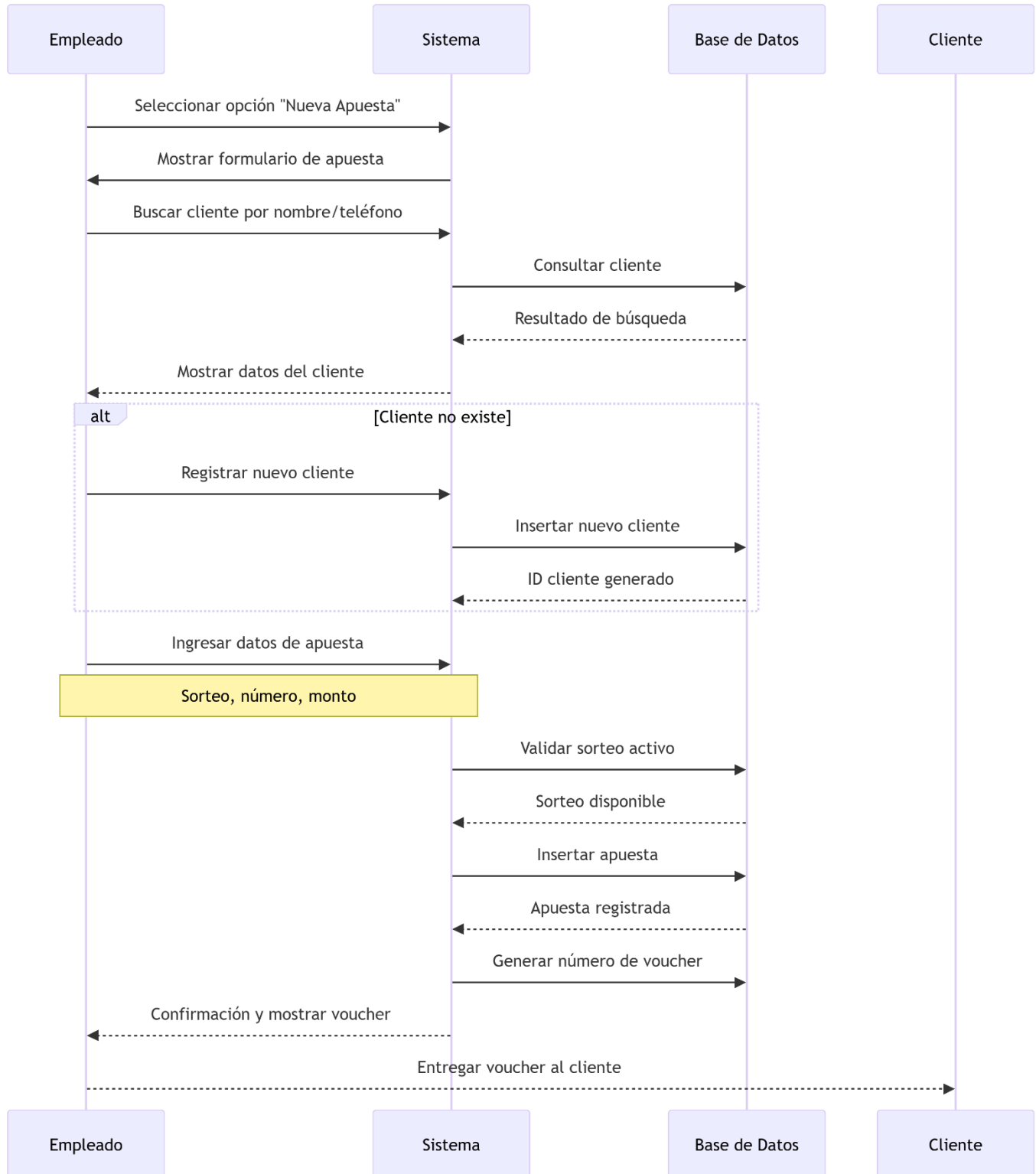
1. Diagrama Entidad-Relación (DER)



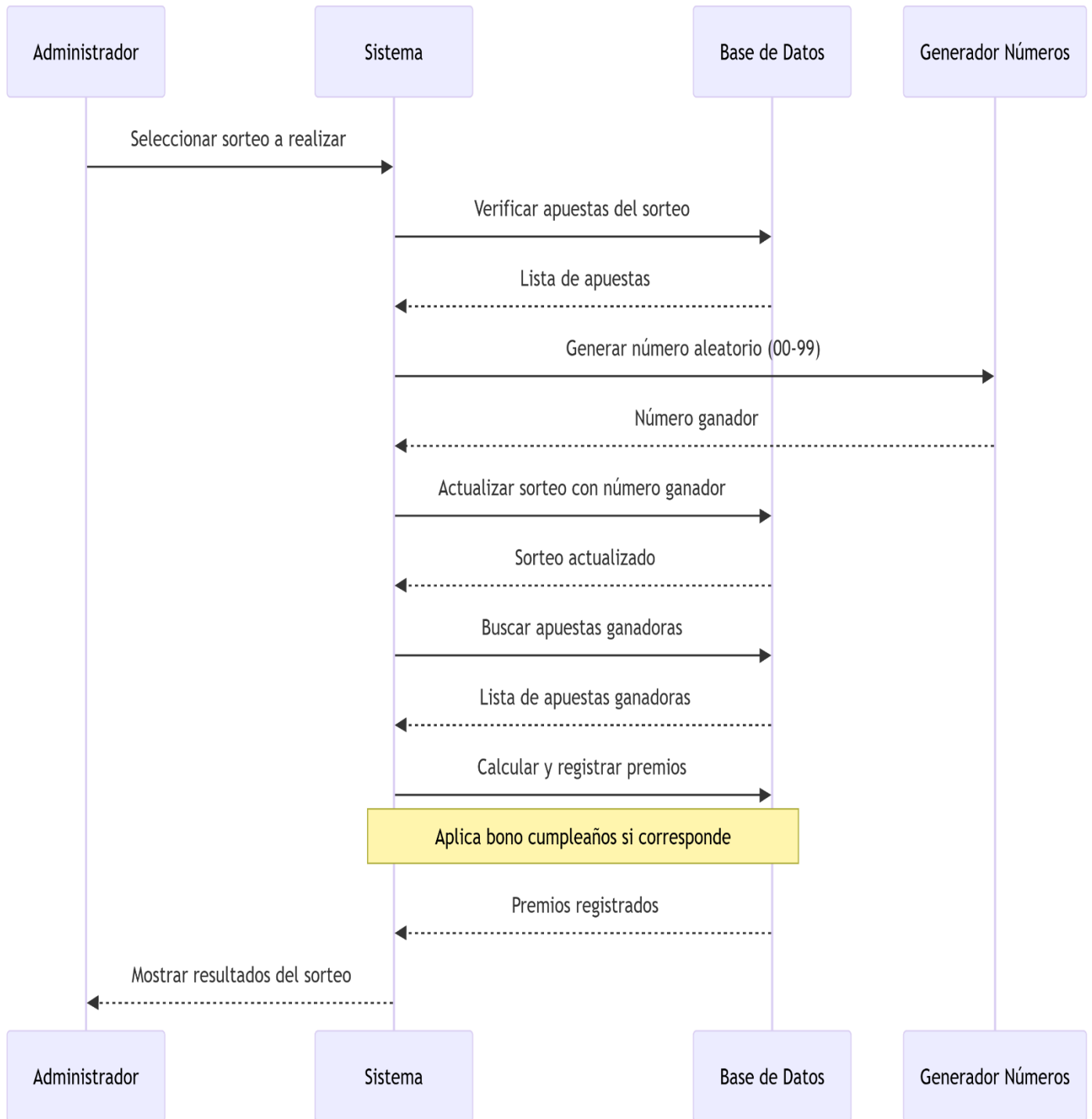
2. Diagrama de Casos de Uso



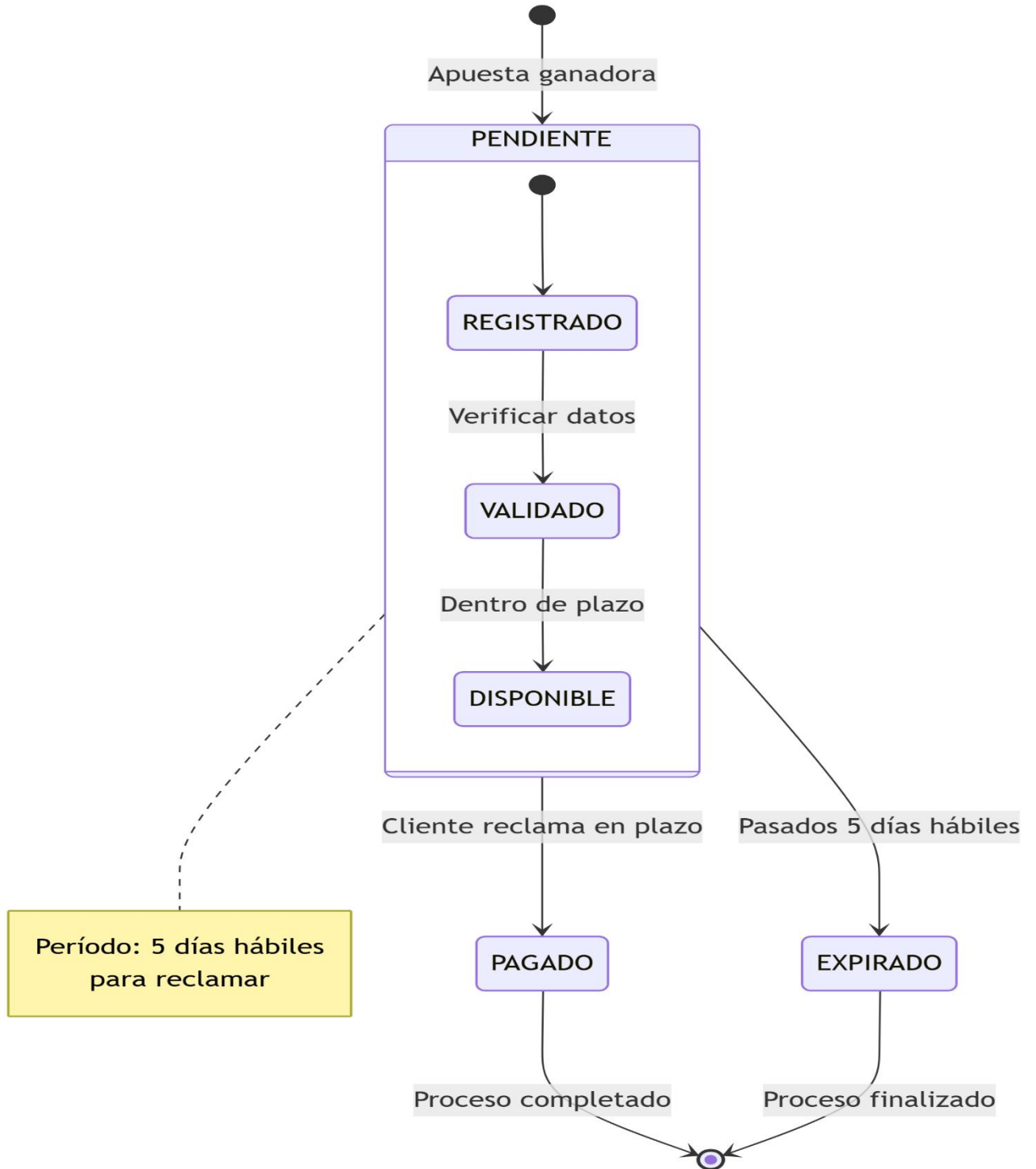
3. Diagrama de Secuencia - Registro de Apuesta



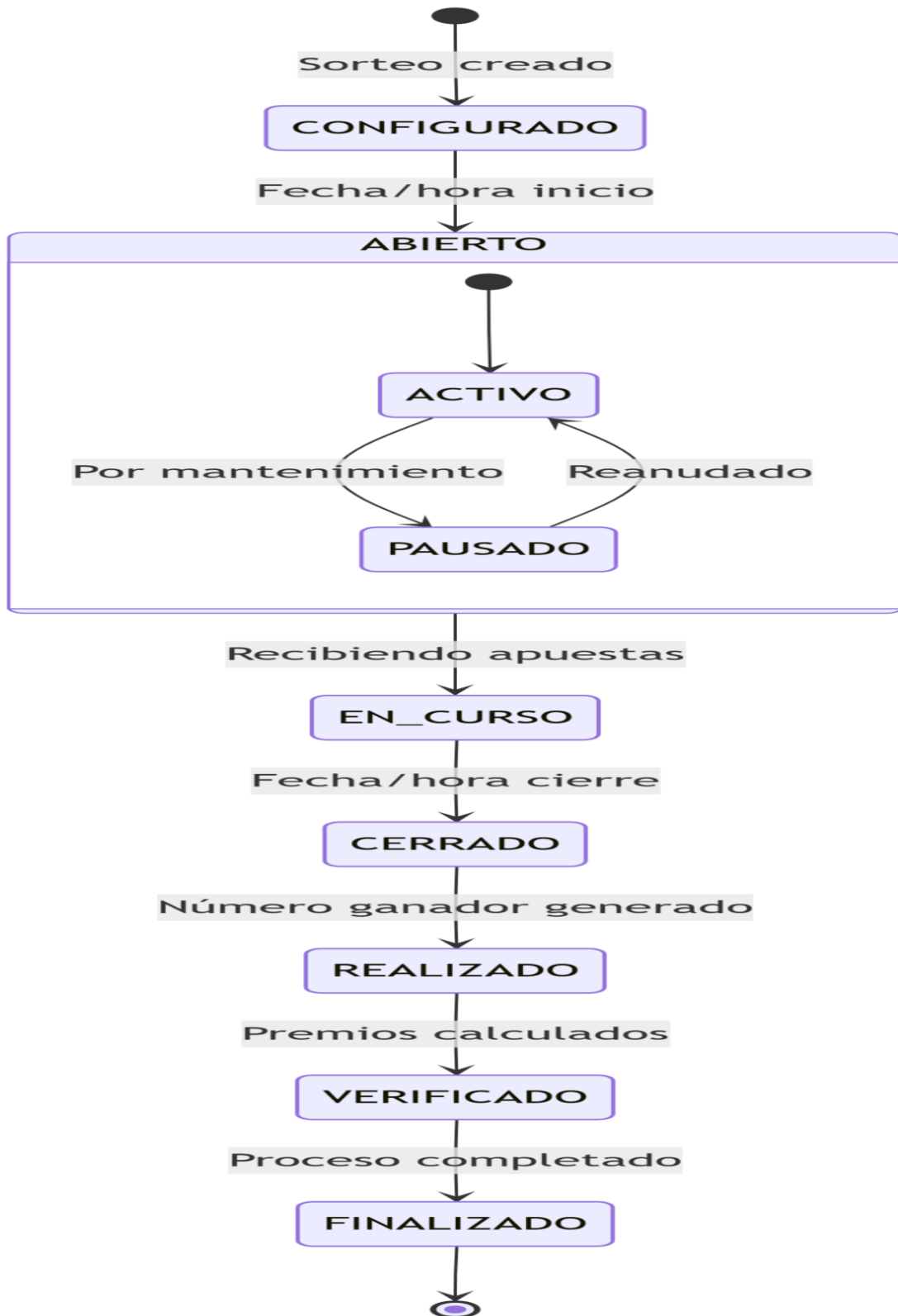
4. Diagrama de Secuencia - Realización de Sorteo



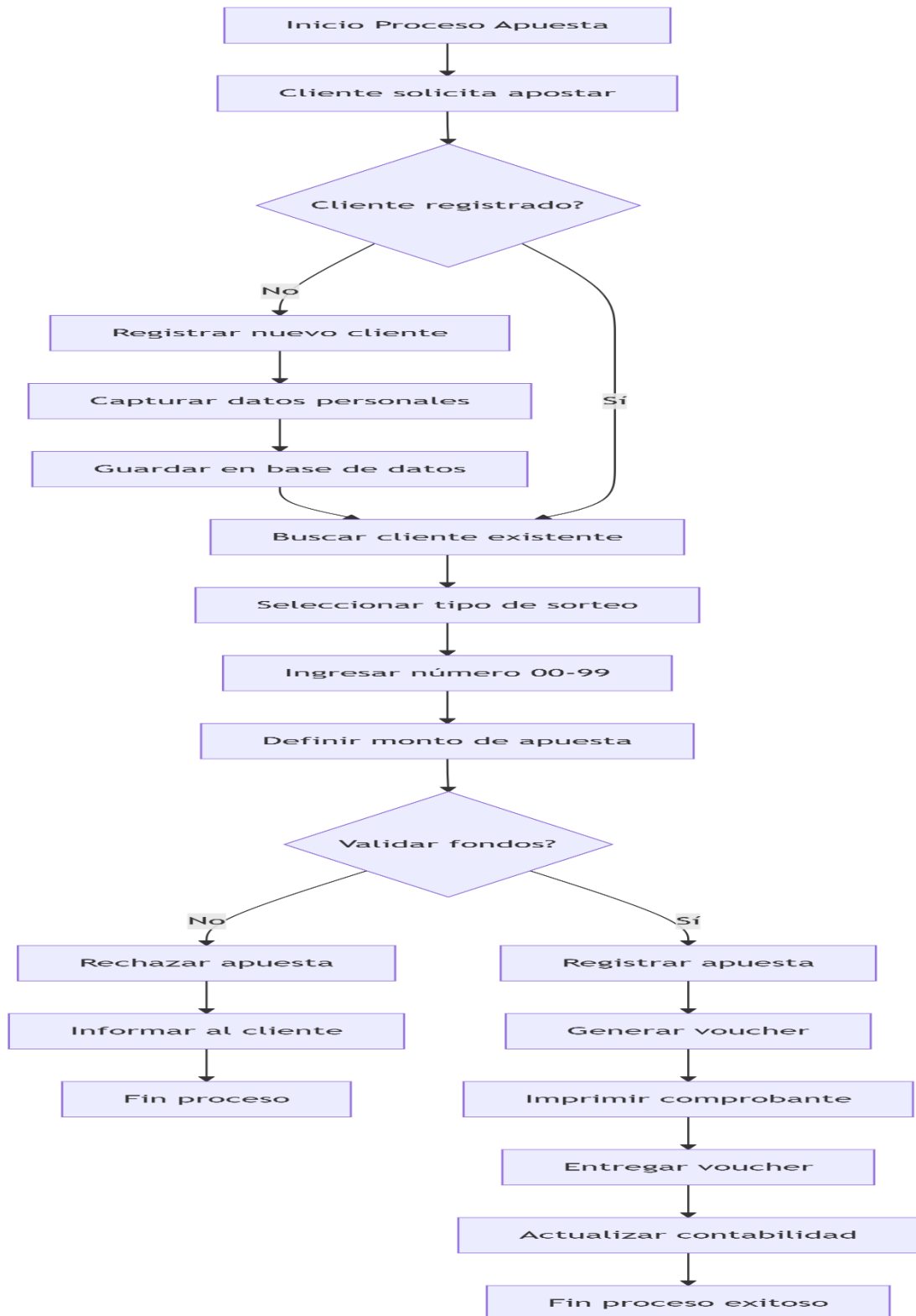
5. Diagrama de Estados - Premio



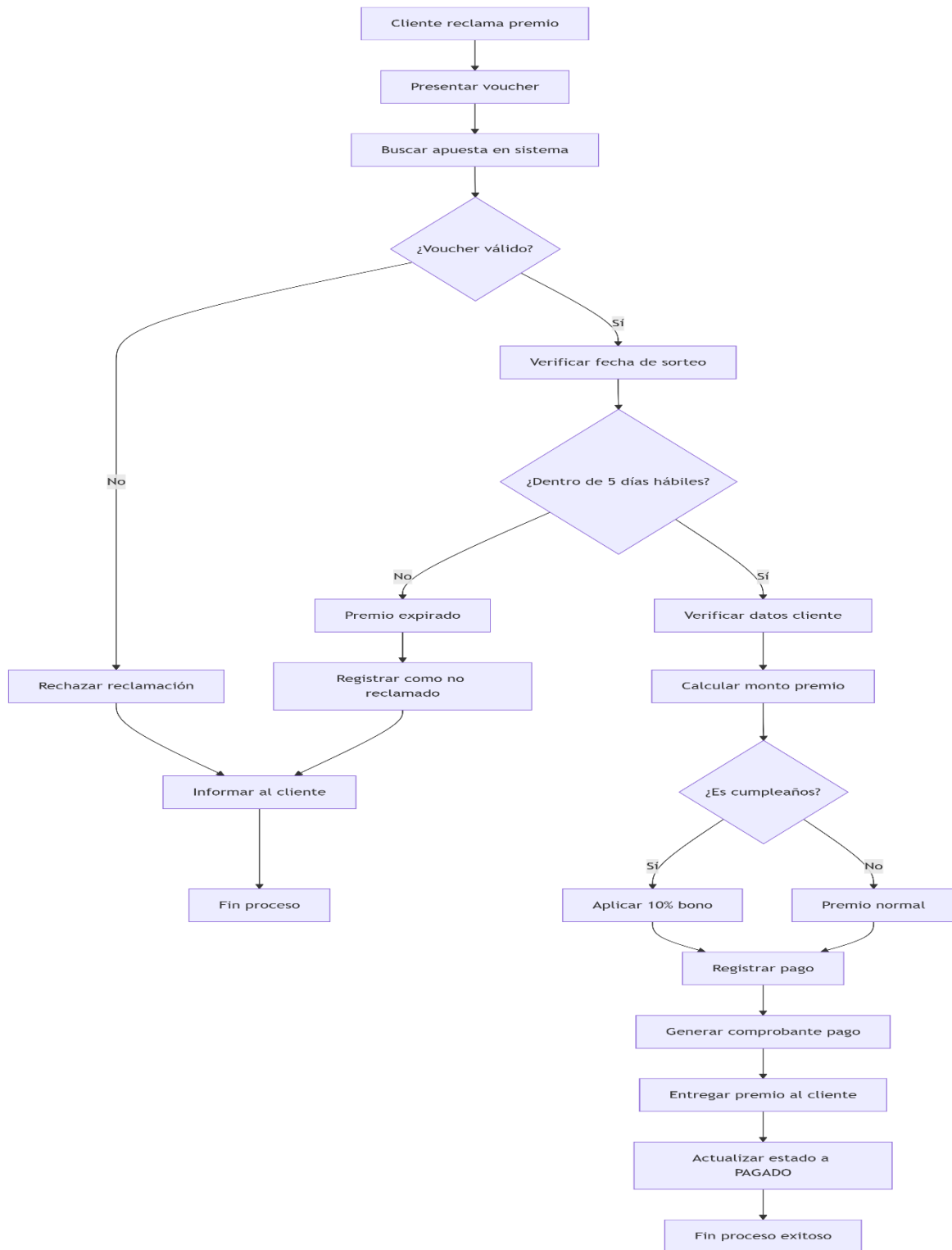
6. Diagrama de Estados - Sorteo



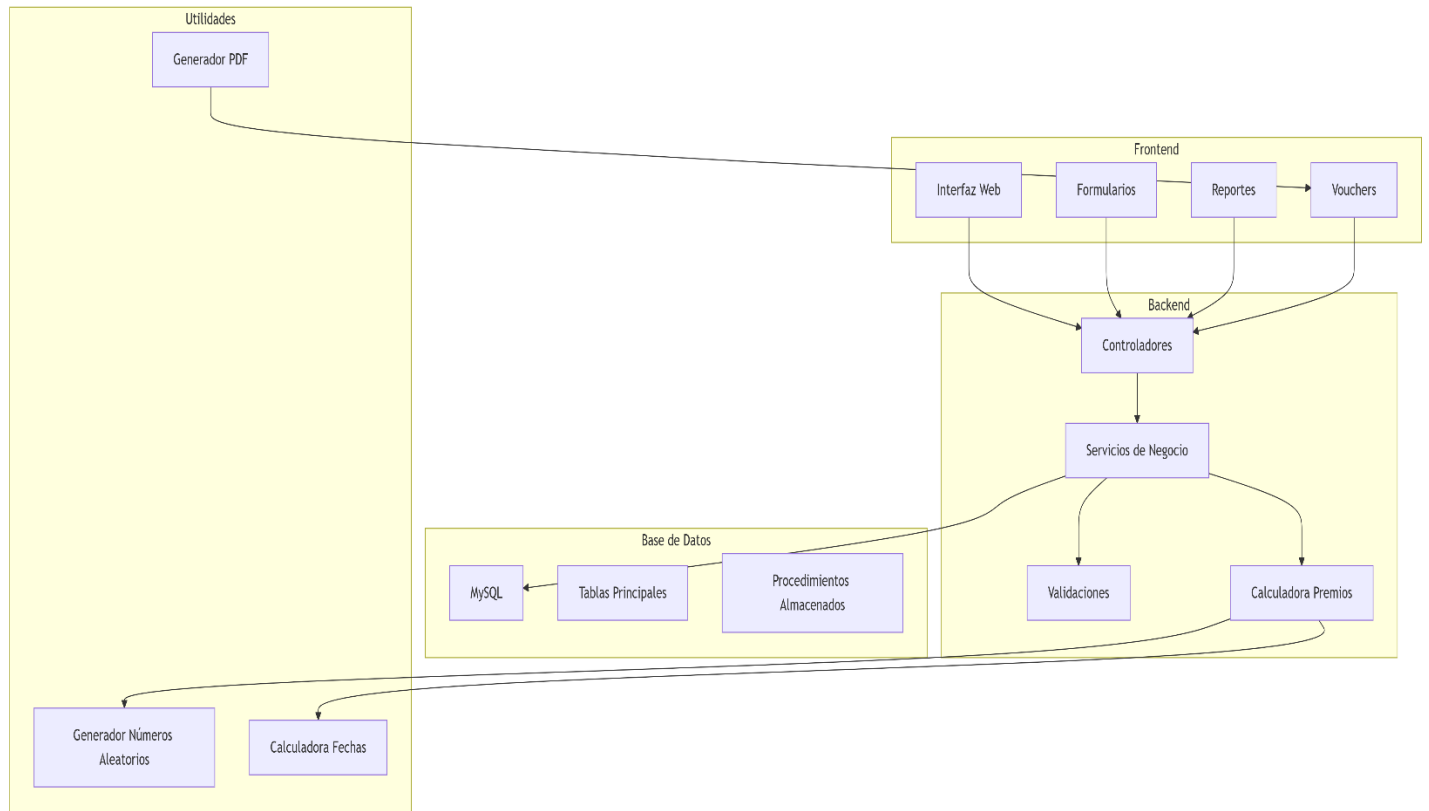
7. Diagrama de Actividades - Proceso Completo de Apuesta



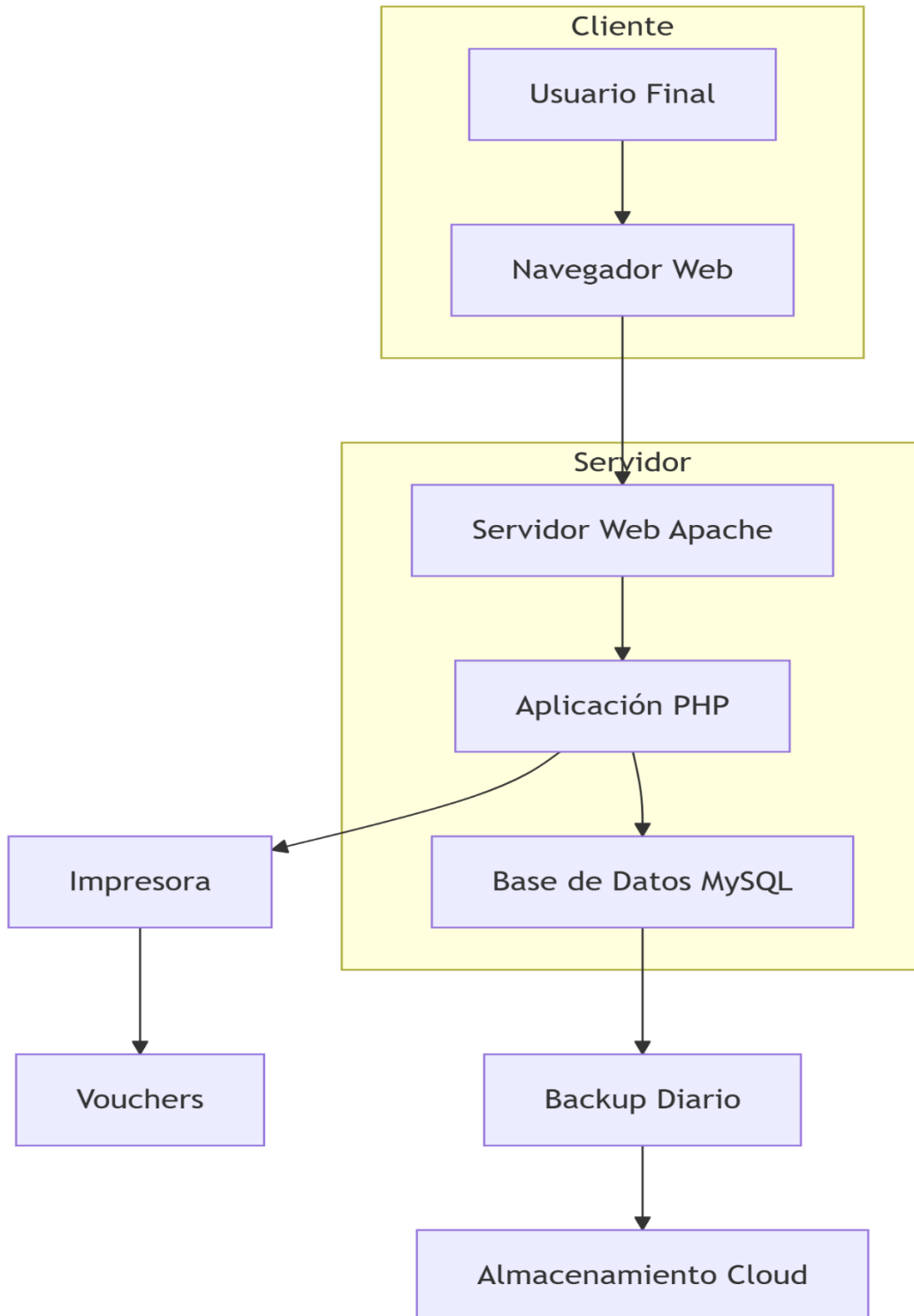
8. Diagrama de Actividades - Proceso de Reclamación de Premio



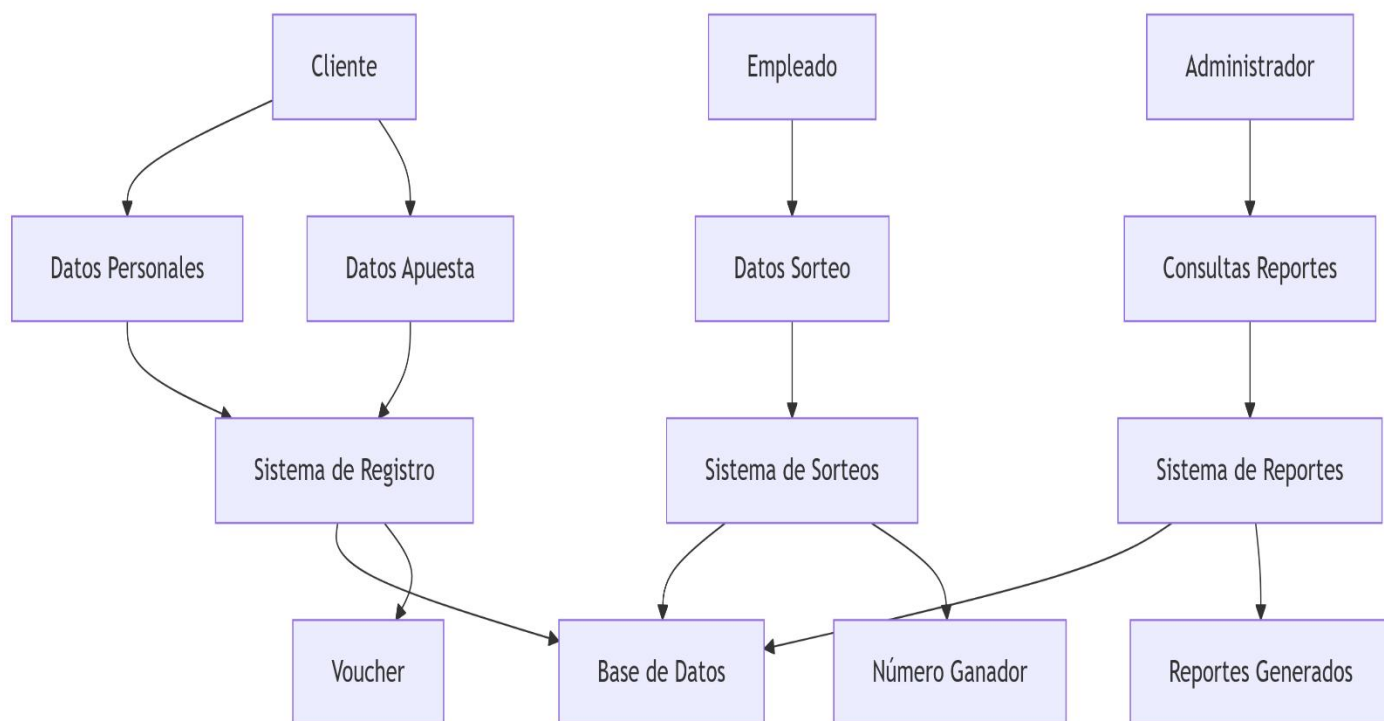
9. Diagrama de Componentes



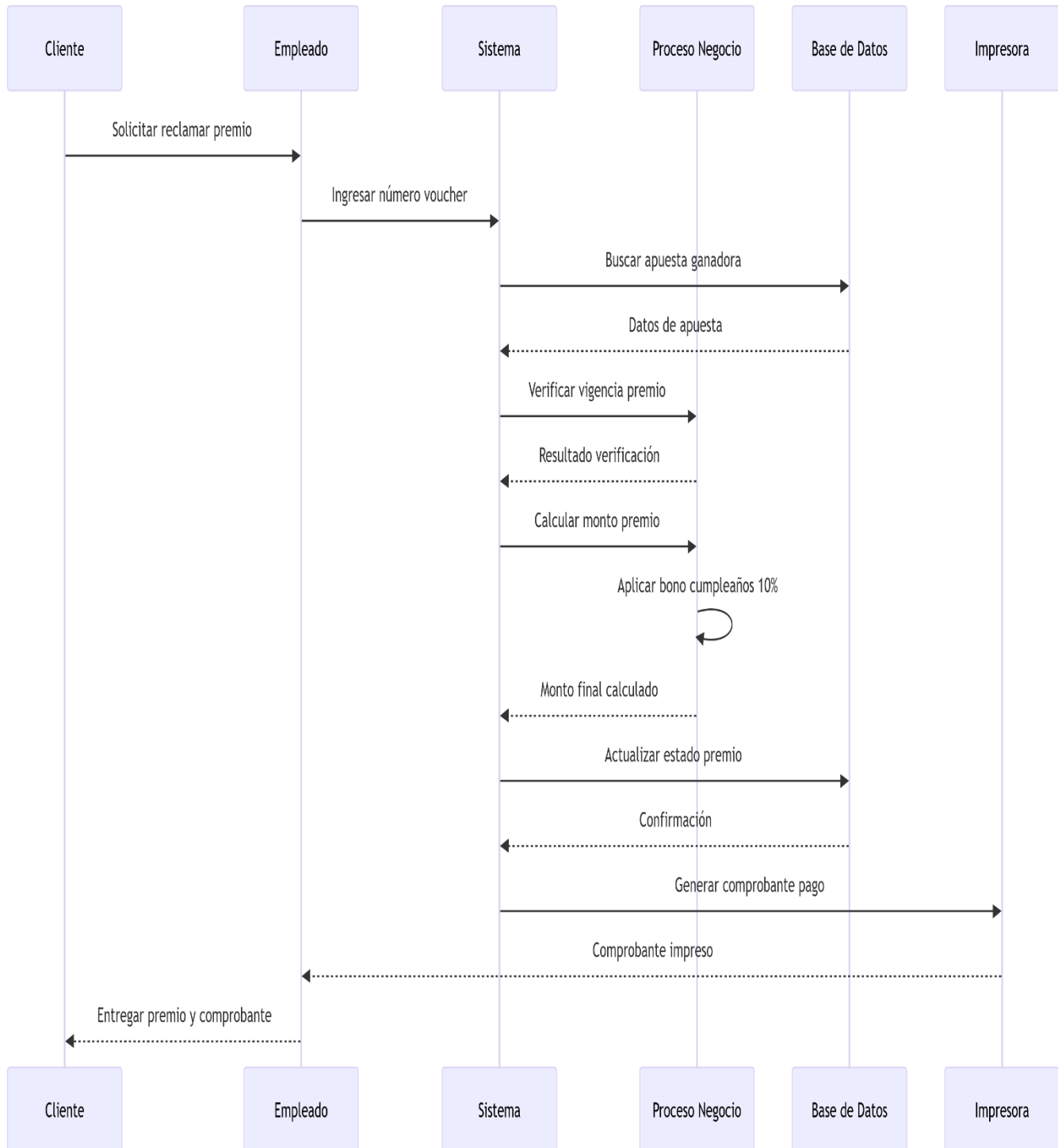
10. Diagrama de Despliegue



11. Diagrama de Flujo de Datos - Nivel 0



12. Diagrama de Comunicación - Sistema de Premios



13. Diagrama de Paquetes

